CS4221

AY25/26 SEM 2

github/Gavino3o

Database Tuning

- PostgreSQL Architecture: SQL Parser and Rewriter → Query Planner and Optimizer → Executor → Memory and Storage Management.
- Planner/Optimizer: Generates query plans using catalogue and statistics, choose plan with least estimated cost.
- Cost is estimated based on the number of rows processed, the number of disk I/O operations, and the CPU time required for the operations.
- Execution Plan: DAG/Tree of operations.
- **Types**: Sequential Scan, Index Scan, Bitmap Index Scan, Bitmap Heap Scan, Nested Loop Join, Hash Join, Merge Join, Aggregate Operation etc.
- Total query time: query planning time + execution time, network latency, client overhead.
- **EXPLAIN (ANALYZE, VERBOSE)** command: shows the execution plan, execution time, row count for each step. (e.g. `EXPLAIN ANALYZE SELECT * FROM Employees WHERE age > 30`)
- Only supports: `SELECT, INSERT, UPDATE, DELETE, EXECUTE, CREATE TABLE, DECLARE`.
- In the query plan: (cost=startup cost..total cost rows=estimated rows width=estimated row width (in bytes)). Startup cost is the cost before the first row is returned, total cost is cost of executing the node (including its children).
- **pg_tables**: Catalog table that contains information about tables in the database. (e.g. `SELECT * FROM pg_tables`)
- **pg_indexes**: Catalog table that contains information about indexes in the database. (e.g. `SELECT * FROM pg_indexes`)
- **pg_stats/pg_statistics**: Catalog table that contains statistics about the distribution of values in the columns of the tables. Populated by the `ANALYZE` command (e.g. `SELECT * FROM pg_stats`)
- **pg_attributes**: Catalog table that contains information about the attributes (columns) of the tables. (e.g. `SELECT * FROM pg_attributes`)
- **Total time of a node** = product of actual time * loops
- **VACUUM**: Reclaims storage occupied by dead tuples. Also updates statistics for the query planner. (e.g. `VACUUM ANALYZE Employees`)
- **Index Types**: B-tree (default, supports equality and range queries), Hash (supports equality queries), GIN (supports full-text search), GiST (supports spatial data), BRIN (supports large tables with natural ordering).
- **Indexes** generally slow down write operations (INSERT, UPDATE, DELETE) because the index also needs to be updated. However, they can significantly speed up read operations (SELECT) by allowing the query planner to quickly locate the relevant rows.
- **PostgreSQL** automatically creates **B-tree unique indexes for primary key and unique constraints**. However, it does not automatically create indexes for foreign keys.
- **SYNTAX**: `CREATE INDEX index_name ON table_name (column_name...);`
- We can create unique indexes. CREATE UNIQUE INDEX...
- We can use WHERE to create partial indexes.
- **Clustering**: Reorganizes the physical storage of the table to match the order of an index. This can improve the performance of queries that use the index. (e.g. `CLUSTER Employees USING idx_age`)
- **A table can only be clustered on one index at a time.** Clustering needs to be **reapplied** after any write operation that modifies the clustered index.
- **Multi-column Indexes**: The order of the columns in the index matters. Index is used if condition involves **prefix** of the multi-column index (e.g. `CREATE INDEX idx_age_name ON Employees (age, name)`)

Data Retrieval Operations

- **Sequential Scan**: Reads the entire table sequentially. Used when there is no index, or when the query planner estimates that it will be faster than using an index (e.g. when a large portion of the table is being scanned).
- **Sort**: Sorts the rows based on the specified columns. Used for ORDER BY, GROUP BY, DISTINCT, etc. (Quick Sort, Merge Sort, etc)
- **Grouping/Aggregation**: Groups the rows based on the specified columns and applies aggregate functions (e.g. SUM, COUNT, AVG) to each group. Used for GROUP BY, aggregate functions without GROUP BY, etc.
- **Unique**: Removes duplicate rows from the result set. Used for DISTINCT, etc. **Note:** Don't use DISTINCT for candidate key columns.
- **Index Only Scan**: Uses an index to retrieve the values of the indexed columns, without accessing the table. Used when index is a covering index.
- **Index Scan**: Uses an index to retrieve the values of the indexed columns, and then accesses the table to retrieve the values of the non-indexed columns. Used when there is an index on the columns used in the WHERE clause, but the index is not a covering index.
- **Bitmap Heap Scan**: Uses a bitmap index scan to retrieve the row locations of the matching rows, and then accesses the table to retrieve the values of the columns. Used when there is an index on the columns used in the WHERE clause, but the index is not a covering index, and the query planner estimates that it will be faster than using an index scan (e.g. when a large portion of the table is being scanned).
- **Bitmap Index Scan**: Uses a bitmap index to retrieve the row locations of the matching rows. Can be used for conjunctions or disjunctions of predicates on indexed columns.

Join Operations

- **Materialization**: Materializes the result of the join into a temporary table. Used when the join is expensive and the result will be reused multiple times.
- **Nested Loop Join**: For each row in the outer table, scan the inner table to find matching rows. Used when one of the tables is small, or when there are indexes on the join columns. The smaller table is usually the outer table.
- **Hash Join**: Builds a hash table on the smaller table (the build input), and then probes the hash table with the rows from the larger table (the probe input) to find matching rows. Used when there are no indexes on the join columns, and the tables are large.
- **Merge Join**: Sorts both tables on the join columns, and then merges the sorted tables to find matching rows. Used when there are no indexes on the join columns, and the tables are large, and the join columns are already sorted (e.g. due to clustering).
- **Semi Join**: Returns rows from the outer table that have a match in the inner table, but does not return any columns from the inner table. Used for `EXISTS, IN` subqueries.
- **Anti Join**: Returns rows from the outer table that do not have a match in the inner table. Used for `NOT EXISTS, LEFT JOIN + IS NULL` subqueries.
- Note: `NOT IN, <> ALL` in PostgreSQL is often done with sequential scan
- **HASH RIGHT/LEFT JOIN**: A variant of hash join that can be used when the smaller table is on the right/left side of the join.
- **Materialized Views**: A materialized view is a precomputed result of a query that is stored as a table. It can be used to speed up queries that are expensive to compute. However, it needs to be refreshed when the underlying data changes.
-

Functional Dependencies

- Functional dependencies are denoted by σ
- Given: $X \to Y$.
  If two tuples of r (a table with relation schema R) agress on their X-values, then they agree on their Y-values.
- Note: SQL query cannot be used in `CHECK` constraints
- Functional dependencies may be enforced when LHS has `PRIMARY KEY` or `UNIQUE` constraints.
- **Trivial** FD: $X \to Y$ where $Y \subseteq X$
- **Non-trivial** FD: $X \to Y$ where $Y \subsetneq X$
- **Completely non-trivial** FD: $X \to Y$ where $Y \neq \emptyset$ and $Y \cap X = \emptyset$
- Course convention allows: $\emptyset \to \emptyset, \emptyset \to Y, X \to \emptyset$
- **Superkey**: X is a superkey if $X \to R$ (i.e. X functionally determines all attributes in R)
- **Candidate key**: X is a candidate key if X is a superkey and there is no proper subset of X that is a superkey. I.e $X \to R$ and $\nexists Y \subsetneq X$ such that $Y \to R$, X is minimal.
- **Logical Entailment**: $F \models f$ if every relation that satisfies F also satisfies f. E.g., if $F = \{X \to Y, Y \to Z\}$, then $F \models X \to Z$.

Armstrong's Axioms

- **Reflexivity**: If $Y \subseteq X$, then $X \to Y$ (i.e. trivial FDs are always valid)
- **Augmentation**: If $X \to Y$, then $XZ \to YZ$ for any Z (You can add the same attributes to both sides of an FD and it still holds)
- **Transitivity**: If $X \to Y$ and $Y \to Z$, then $X \to Z$ (i.e. You can short circuit the arrows)
- **Union Rule**: If $X \to Y$ and $X \to Z$, then $X \to YZ$
- **Decomposition Rule**: If $X \to YZ$, then $X \to Y$ and $X \to Z$
- **Pseudotransitivity**: If $X \to Y$ and $YW \to Z$, then $XW \to Z$
- **Sound**: If an FD can be derived from Armstrong's Axioms, then it is logically entailed by the set of FDs.
- **Complete**: If an FD is logically entailed by a set of FDs, then it can be derived from Armstrong's Axioms.

Closure of FDs and Attributes

- The closure of a set of functional dependencies $\sum$ (denoted $\sum^+$) is the set of all FDs that can be logically entailed by $\sum$.
- $\sum^+ = \{X \to Y | \sum \models X \to Y\}$
- **Attribute Closure**: The closure of a set of attributes X under a set of FDs $\sum$ (denoted X^+) is the set of all attributes that are functionally determined by X under $\sum$.
- $X^+ = \{A | \exists (X \to \{A\}) \in \sum^+\}$
- **Algorithm (1) to compute Attribute Closure**:
  1. Initialize $X^+ = X$
  2. For each FD $Y \to Z$ in $\sum$, if $Y \subseteq X^+$, then add Z to X^+
  3. Repeat step 2 until no more attributes can be added to X^+
- **Equivalence of FD sets**: Two sets of FDs $\sum$ and $\sum'$ are equivalent if $\sum^+ = \sum'^+$ (i.e. they have the same closure).
- To check for equivalence, check if they cover each other:
  1. For each FD in $\sum_A$, use FDs in $\sum_B$ to compute the attribute closure of the LHS of the FD.
  2. If the closure contains the RHS, then the FD is implied by $\sum_B$.
  3. If all FDs in $\sum_A$ are implied by $\sum_B$, then $\sum_A$ is implied by $\sum_B$.
  4. Repeat the same process to check if $\sum_B$ is implied by $\sum_A$. If both are implied, then $\sum_A$ and $\sum_B$ are equivalent.

Minimal Cover

- A set of FDs $\sum$ is a minimal if:
  1. The **RHS** of every FD in $\sum$ is a single attribute (i.e. FDs are in canonical form)
  2. The **LHS** of every FD in $\sum$ is minimal (i.e. there are no extraneous attributes in the LHS)
  3. There are **no redundant** FDs in $\sum$ (i.e. there is no FD in $\sum$ that can be removed without changing the closure of $\sum$)
- **Algorithm (2) to compute minimal cover**:
  1. Decompose FDs so that the RHS of every FD is a single attribute (i.e. put FDs in canonical form)
  2. For each FD, check if there are extraneous attributes in the LHS. If there are, remove them. To check if an attribute A in the LHS of an FD $X \to Y$ is extraneous, compute the attribute closure of $X - A$ under the set of FDs (including $X \to Y$). If the closure contains Y, then A is extraneous and can be removed from the LHS.
  3. For each FD, check if it is redundant. To check if an FD $X \to Y$ is redundant, compute the closure of X under the set of FDs without $X \to Y$. If the closure contains Y, then $X \to Y$ is redundant and can be removed from the set of FDs.
- Minimal cover is not unique. There can be multiple minimal covers for a given set of FDs.
- **Compact:** There is no FD with the same LHS.
- **Canonical Cover**: Compact and minimal cover.

The Chase Algorithm

- Test whether a give FD f is logically entailed by a set of FDs $\sum$.
- Construct a tableau with two rows and n columns, where n is the number of attributes in the relation schema.
- The first row is filled with α and the second row filled with distinct elements.
- For the FD $f : X \to Y$, replace the elements in the second row with α for all attributes in X.
- Apply the FDs in $\sum$ to the tableau by changing distinct elements into α until no more changes can be made. Note: LHS column α.

Normalization

- **Prime Attribute**: An attribute that is part of a candidate key.
- **1NF**: A relation is in 1NF if all attributes are atomic (i.e. indivisible). There are no repeating groups or arrays in the relation.
- **2NF**: A relation is in 2NF if it is in 1NF and there are no partial dependencies (i.e. no non-prime attribute is functionally dependent on a proper subset of any candidate key). $X \to \{A\}$ is trivial or A is a prime attribute.
- **3NF**: A relation is in 3NF if it is in 2NF and there are no transitive dependencies ($X \to \{A\}$ is trivial or A is a prime attribute or X is a superkey).
- **BCNF**: A relation is in BCNF if for every non-trivial FD $X \to Y$, X is a superkey. Note: BCNF is a stronger condition than 3NF.
- **4NF**: A relation is in 4NF if for every non-trivial multivalued dependency $X \twoheadrightarrow Y$, X is a superkey. Note: 4NF is a stronger condition than BCNF.

Decomposition

- **Losslessness**: Reconstructablity. A decomposition is lossless-join decomposition if the **natural join** of all the framents is equivalent to the original relation. (i.e. no spurious tuples are generated when we join the fragments)
- **Dependency Preservation**: A decomposition is dependency preserving if the **union of the projections of the FDs** onto the fragments is equivalent to the original set of FDs (i.e. we can enforce all the FDs by enforcing them on the fragments).
- **Lemma 1, Binary Decomposition**: A decomposition of R into R_1 and R_2 is lossless if $R_1 \cap R_2 \to R_1$ or $R_1 \cap R_2 \to R_2$ (i.e. the common attributes of the two fragments functionally determine one of the fragments).
- **Lemma 2, General Decomposition**: A decomposition of R into $R_1, R_2, ..., R_n$ is lossless if there exists at least one sequence of binary lossless-join decomposition that generates the decomposition.

Chase Algorithm

- To check if a decomposition is lossless.
- **Input**: A relation schema R, a set of FDs $\sum$, and a decomposition of R into $R_1, R_2, ..., R_n$.
  1. Create a table r with schema R with n tuples, where n is the number of fragments in the decomposition.
  2. For each attribute A in R_i, set the A values in the i^{th} tuple as α, treat the rest as distinct symbols.
  3. For each FD $X \to Y$ in $\sum$, if the X values in a tuple are the same, then set the Y values in that tuple to be the same. Same α only.
  4. Chase to find a row of all α's.

Projected Set of FDs

- Way to compute the projeted set of FDs on a fragment R_i given the original set of FDs $\sum$.
  1. Given attribute C is missing
  2. If C appears in the **RHS** of an FD, remove that FD.
  3. If C appears in the LHS of an FD, remove C from the LHS of that FD.
  4. Care for transitivity: If C is in the LHS of an FD and there is another FD where C is in the RHS, we need to check if the transitive dependency can be preserved. If it cannot be preserved, we need to add a new FD with the transitive dependency.

Decomposition Algorithm

- **BCNF Decomposition Algorithm**
  1. **Initialise:** Start with the set of relations $D = \{R\}$.
  2. **Check:** While there is a relation $R_i \in D$ not in BCNF:
     - Find a violation $X \to Y$ (where $X \to Y \in \Sigma^+$, $Y \not\subseteq X$, and X is not a superkey for R_i).
     - Compute the attribute closure X^+ with respect to Σ.
     - **Split** R_i into two new relations:
       2.1. $R_{i_1} = X^+$
       2.2. $R_{i_2} = (R_i - X^+) \cup X$
     - Replace R_i in D with R_{i_1} and R_{i_2}.
  3. **Result:** D is a lossless-join decomposition of R into BCNF.
- **Note:** The BCNF decomposition algorithm does not guarantee dependency preservation.
- **3NF Synthesis Algorithm**
  1. **Candidate keys:** Identify candidate keys of R using attribute closure.
  2. **Canonical Cover:** Compute the minimal cover Σ^- of Σ, and compact them.
  3. **Group:** For each $X \to Y \in \Sigma^-$, create a relation $R_i = X \cup Y$.
  4. **Merge:** If two relations R_1, R_2 have keys X, Z such that $X \to Z$ and $Z \to X$, merge them.
  5. **Key Check:** If no R_i contains a candidate key of R, add a new relation R_{key} consisting of any candidate key of R.
  6. **Purge:** Remove any relation R_i that is a subset of another relation R_j.
- **Note:** The 3NF synthesis algorithm guarantees both *lossless-join and dependency preservation.* May find BCNF.
- $R = \{A, B, D, E\}$ with $\sum = \{BE \to AD, D \to E\}$. Use another table to enforce $D \to E$ via foreign key.